

```
In [4]: #!/pip install scikit-fem
import skfem as fem
from skfem import BilinearForm, LinearForm, asm
from skfem.helpers import dot, grad
from skfem.models import laplace, mass

import numpy as np
import matplotlib.pyplot as plt
from scipy.sparse import csr_matrix, identity
from scipy.sparse.linalg import spsolve, splu

import imageio
```

Population Dynamics Model with Migration and Diffusion

The article studies population dynamics in a one-dimensional domain. The dynamics are governed by the transient (i.e. that marks the evolution between two equilibria), non-linear, advection-diffusion-reaction equation (1), which allows to model the propagation of population fronts for different initial conditions.

This model allows to account for both a random (diffusive) and directed (advective, i.e. transfer of matter through the movement of a fluid) origin of propagation, a biologically justified approach which has been first explored by Banks et al. (1988): a population may move by choice, e.g. an insect flying upwind towards a patch of odourant flowers or to avoid a predator.

Let Ω be an open region in $\mathfrak{R}^1$ with a piecewise smooth boundary Γ . The equation is:

$$\frac{\partial U}{\partial T} = -\frac{\partial}{\partial X} \left[V(U)U - D \frac{\partial U}{\partial X} \right] + F(U)U \quad \text{in } \Omega, T > 0 \quad (1)$$

Here:

- $U = U(X, T)$ is the unknown population density field at position $X \in \Omega$ and time $T \in \bar{I} \equiv (0, T)$ (individuals per unit space).
- $V \equiv V(U)$ is the migration-advection velocity field (directed movement).
- D is the (Fickian) diffusion coefficient, showing how individuals spread out randomly (random movement).
- $F(U)$ is the growth rate, including birth, death, and trophic relationships (local processes).

This equation can describe how a population front moves :

- **Invasion:** the population spreads into a new area where it did not exist, the density increases with time in a given space.
- **Retreat :** the population decreases in a given space, and "moves back" to the area where it was already present.

The advection field (2) is assumed to be non-linear and to be influenced by two parameters:

- V_0 , the **density-independent migration speed**, a constant background velocity and may originate from hydrodynamic models and it is unaffected by population density.
- V_1 , the **density-dependent migration**, which changes linearly with population density, and represents species taxis (i.e. an orientation movement of free organisms, triggered by an external stimulus).

$$V(U) = V_0 + V_1 U \quad (2)$$

In this migration model, for simplicity, it is assumed that:

- the fluid is **incompressible**, i.e. $\text{div}(V_0) = 0$,
- V_0 , V_1 , and D (the diffusion coefficient) are **constants**.

Then the population model becomes:

$$\begin{aligned} \frac{\partial U}{\partial T} &= -\frac{\partial}{\partial X} \left[V(U)U - D \frac{\partial U}{\partial X} \right] + F(U)U \\ \frac{\partial U}{\partial T} &= -\frac{\partial}{\partial X} \left[V_0 U + V_1 U^2 - D \frac{\partial U}{\partial X} \right] + F(U)U \\ \frac{\partial U}{\partial T} &= -V_0 \frac{\partial U}{\partial X} - V_1 \frac{\partial U^2}{\partial X} + D \frac{\partial^2 U}{\partial X^2} + F(U)U \\ \frac{\partial U}{\partial T} + V_0 \frac{\partial U}{\partial X} + V_1 \frac{\partial U^2}{\partial X} &= D \frac{\partial^2 U}{\partial X^2} + F(U)U \\ \frac{\partial U}{\partial T} + V_0 \frac{\partial U}{\partial X} + 2V_1 U \frac{\partial U}{\partial X} &= D \frac{\partial^2 U}{\partial X^2} + F(U)U \\ \frac{\partial U}{\partial T} + (V_0 + 2V_1 U) \frac{\partial U}{\partial X} &= D \frac{\partial^2 U}{\partial X^2} + F(U)U \end{aligned} \quad (3)$$

Following Lewis and Kareiva (1993), the growth rate with the Allee effect is defined as:

$$F(U)U = \bar{\alpha}U(U - K_0)(K - U) \quad (4)$$

Where:

- K is the **carrying capacity**: maximum sustainable population.
- K_0 measures the **Allee effect strength**

Since K is constant, the function is then normalised and made adimensional by setting $u = \frac{U}{K}$:

$$f(U) = \alpha u(u - \beta)(1 - u) \quad (5)$$

Where:

- $\beta = \frac{K_0}{K}$ represents the **intensity of the Allee effect** (i.e. its strength normalised with respect to the carrying capacity).
- $\alpha = \alpha(\beta)$ is a **normalization constant** based on the maximum growth rate. Lewis and Kareiva (1993) showed that the qualitative results regarding the Allee effect and the rate of spread do not depend on the choice of the value of this constant.

Allee Effect Classification

- **Strong Allee** : $0 < \beta < 1$ positive density dependance at low densities.
- **Weak Allee** : $-1 < \beta < 0$ reduced but positive growth at low densities.

Inputting (5) in (3) and using the dimensionless variables t, x and parameters v_0, v_1 defined as follows

- $t = T\alpha K^2,$
- $x = X\sqrt{\frac{\alpha K^2}{D}},$
- $v_0 = \frac{V_0}{K\sqrt{\alpha D}},$
- $v_1 = \frac{2V_1}{\sqrt{\alpha D}},$

equation (3) becomes:

$$\frac{\partial u}{\partial t} + (v_0 + v_1 u) \frac{\partial u}{\partial x} = \frac{\partial^2 u}{\partial x^2} - \beta u + (1 + \beta)u^2 - u^3 \quad (6)$$

With these parameters, the **population density** has been rescaled so that: $u \in [0, 1]$ in $t \in I$.

We now consider **travelling wave solutions**, solving the dimensionless equation (6) in an unbounded domain, with the following boundary conditions at infinity:

$$\begin{aligned} \text{for } x \rightarrow +\infty &\Rightarrow u = 1 \quad (\text{the species is at its carrying capacity}) \\ \text{for } x \rightarrow -\infty &\Rightarrow u = 0 \quad (\text{the species is absent}) \end{aligned} \quad (7)$$

From a computational point of view, we use a **bounded domain** large enough to approximate these conditions, such that:

- $u = g_1 = 1$ at the upper boundary ($x \rightarrow +\infty$)
- $u = g_0 = 0$ at the lower boundary ($x \rightarrow -\infty$)

Under these boundary conditions, **Fife (1979)** and **Aronson & Weinberger (1978)** analyzed the **asymptotic stability** of the travelling wave for the scaled diffusion–reaction equation:

$$\frac{\partial u}{\partial t} = \frac{\partial^2 u}{\partial x^2} + g(u) \quad (8)$$

Where: $g(u) = -\beta u + (1 + \beta)u^2 - u^3$

to analyze wave propagation, we transform to a **moving coordinate system**:

$$u(x, t) = U(\xi) \quad \text{where} \quad \xi = x - ct$$

where:

- ξ : Moving coordinate frame
- c : Wave speed
- $U(\xi)$: Wave profile function

Derivative Transformations

- **Time derivative:** $\frac{\partial u}{\partial t} = -c \frac{dU}{d\xi}$
- **Spatial derivative:** $\frac{\partial u}{\partial x} = \frac{dU}{d\xi}$
- **Second derivative:** $\frac{\partial^2 u}{\partial x^2} = \frac{d^2 U}{d\xi^2}$

Equation in moving Frame

Substituting into the dimensionless equation (8) yields:

$$\frac{\partial^2 U}{\partial \xi^2} + c \frac{\partial U}{\partial \xi} + g(U) = 0 \quad (\text{A})$$

Theoretical Results

According to **Lewis and Kareiva (1993)**, the following conditions determine the convergence toward a **travelling wave solution**.

1. A sufficient condition for convergence to the travelling wave solution $0 < \beta < 1$ is:

Integral calculation of $g(u)$:

$$\int_0^1 g(U) \partial U = \int_0^1 [-\beta U + (1 + \beta)U^2 - U^3] \partial U = \left[-\frac{\beta}{2} U^2 + \frac{1 + \beta}{3} U^3 - \frac{1}{4} U^4 \right]_0^1 =$$

$$\textbf{Wave existence condition: } \frac{1}{12} - \frac{\beta}{6} > 0 \Rightarrow \beta < \frac{1}{2}$$

This leads to the initial data conditions:

$$\lim_{x \rightarrow -\infty} \sup u(x, 0) < \beta \quad \text{and} \quad \lim_{x \rightarrow +\infty} \inf u(x, 0) > \beta \quad (9)$$

When $\beta \leq 0$, the initial conditions must have bounded spatial support. This ensures that the population is initially confined to a finite region of space.

2. For $0 < \beta < 1$:

Method using partial derivatives: Multiply (A) by $\frac{\partial U}{\partial \xi}$:

$$\frac{\partial U}{\partial \xi} \frac{\partial^2 U}{\partial \xi^2} + c \left(\frac{\partial U}{\partial \xi} \right)^2 + g(U) \frac{\partial U}{\partial \xi} = 0$$

Recognizing derivative:

$$\frac{\partial}{\partial \xi} \left[\frac{1}{2} \left(\frac{\partial U}{\partial \xi} \right)^2 + G(U) \right] + c \left(\frac{\partial U}{\partial \xi} \right)^2 = 0$$

where $G(U) = \int g(U) \partial U = -\frac{\beta}{2} U^2 + \frac{1+\beta}{3} U^3 - \frac{1}{4} U^4$

Integration over ξ with boundary conditions: $\frac{\partial U}{\partial \xi}(\pm\infty) = 0, G(1) - G(0) = 0$ gives:

$$c \int_{-\infty}^{\infty} \left(\frac{\partial U}{\partial \xi} \right)^2 \partial \xi = 0$$

Test function solution: $U(\xi) = \frac{1}{1+e^{k\xi}}$ substituted into (A) yields:

$$c = \sqrt{2} \left(\beta - \frac{1}{2} \right) \quad (10)$$

Thus, the direction of propagation can be either positive or negative:

- For $\beta < \frac{1}{2}$, the front propagates into the region where the species is absent, corresponding to **species invasion**.
- For $\beta > \frac{1}{2}$, the front propagates into the region where the species is at its carrying capacity, corresponding to **species retreat**.
- When $\beta = \frac{1}{2}$, the front is **stationary**.

3. For $-1 < \beta \leq 0$:

Linear analysis for $-1 < \beta \leq -\frac{1}{2}$: Equation (A) linearized near $U = 0$:

$$\frac{\partial^2 U}{\partial \xi^2} + c \frac{\partial U}{\partial \xi} - \beta U = 0$$

Exponential solution: $U(\xi) \sim e^{\lambda \xi}$ with $\lambda = \frac{-c \pm \sqrt{c^2 - 4\beta}}{2}$

Maximum speed from spectral analysis: here exists a maximal wave speed c^* such that a wave front solution exists with $c \leq c^*$.

$$c^* = 2\sqrt{-\beta} \quad \text{for} \quad -1 < \beta \leq -\frac{1}{2}$$

Transition and continuity: For $-\frac{1}{2} < \beta \leq 0$, the bound is given by equation (10)

- For $-1 < \beta \leq -\frac{1}{2}$, c^* is derived by linearizing equation (8) about zero, yielding $c^* = 2\sqrt{-\beta}$
- For $-\frac{1}{2} < \beta \leq 0$, the bound is given by equation (10). In this range, the direction of propagation is always **negative**, independent of the strength of the Allee effect. In other words, a weak Allee effect is **not sufficient to prevent invasion**.

Regarding **strong Allee effects** and following an analytical approach, **Petrovskii and Li (2003)** derived exact travelling wave solutions for the general nonlinear equation (6). They obtained the same asymptotic wave speed as in equation (10) for the no-

migration case and also determined the asymptotic wave speed for either **density-independent** or **density-dependent** migrations.

Since the exact solutions were restricted to $\beta \geq 0$, the dynamics with a **weak Allee effect** were not addressed.

In the following, we present a **numerical method** to solve equation (6). It is noteworthy that if V_0, V_1, D or K are not constants, the dimensionless form (6) must be replaced by equation (1). Nevertheless, the general methodology developed here can still be applied.

Model

Approximated solution

In this part, we will describe the functions used to run the finite element method. Since FEM was not covered in our curriculum, it was decided to use a python library, `scikit-fem` ([link to the documentation](#)), to simplify the implementation and not do it all by hand.

As a reminder, we try to solve the following equation in:

$$\frac{\partial u}{\partial t} + v(u) \frac{\partial u}{\partial x} = \frac{\partial^2 u}{\partial x^2} - f_\beta(u) \quad \text{in } \Omega \quad (6)$$

Where Ω is an open region in $\mathcal{R}^1$ domain with piecewise smooth boundary Γ , $f_\beta : u \mapsto \beta u - (1 + \beta)u^2 + u^3$ and $v(u) = v_0 + v_1 u$.

One can move from this strong formulation to the weak formulation, defining the Dirichlet space $W = H_0^1(\Omega)$ and the weight function $w \in W$:

$$\begin{aligned} \frac{\partial u}{\partial t} w + v(u) \frac{\partial u}{\partial x} w &= \frac{\partial^2 u}{\partial x^2} w - f_\beta(u) w \quad \text{in } \Omega, \forall w \in W \\ \int_{\Omega} \frac{\partial u}{\partial t} w dx + \int_{\Omega} v(u) \frac{\partial u}{\partial x} w dx &= \int_{\Omega} \frac{\partial^2 u}{\partial x^2} w dx - \int_{\Omega} f_\beta(u) w dx \quad \text{in } \Omega, \forall w \in W \end{aligned}$$

Using partial differentiation, and especially the following

$$\int_{\Omega} \frac{\partial^2 u}{\partial x^2} w dx = \left[\frac{\partial u}{\partial x} w \right]_{\partial\Omega} - \int_{\Omega} \frac{\partial u}{\partial x} \frac{\partial w}{\partial x} dx$$

the former function can be rewritten as

$$\int_{\Omega} \frac{\partial u}{\partial t} w dx + \int_{\Omega} v(u) \frac{\partial u}{\partial x} w dx = \left[\frac{\partial u}{\partial x} w \right]_{\partial\Omega} - \int_{\Omega} \frac{\partial u}{\partial x} \frac{\partial w}{\partial x} dx - \int_{\Omega} f_\beta(u) w dx \quad \text{in } \Omega,$$

Since w belongs to a Dirichlet space H_0^1 , w vanishes on $\partial\Omega$ and the boundary term $\left[\frac{\partial u}{\partial x} w \right]_{\partial\Omega}$ disappears, and, by moving the right-hand member to the left-hand side, the

following form can be retrieved:

$$\int_{\Omega} \frac{\partial u}{\partial t} w dx + \int_{\Omega} v(u) \frac{\partial u}{\partial x} w dx + \int_{\Omega} \frac{\partial u}{\partial x} \frac{\partial w}{\partial x} dx + \int_{\Omega} f_{\beta}(u) w dx = 0 \quad \text{in } \Omega, \forall w \in W$$

Defining $V = H_g^1(\Omega)$, astutely establishing $L^2(\Omega)$ inner product as

$(u, w) := \int_{\Omega} u w dx$ for $u, w \in L^2(\Omega)$, and tweaking a bit the presentation we finally retrieve the article's weak formulation, which reads: find $u \in V$, for a given t , satisfying

$$\left(\frac{\partial u}{\partial t}, w \right) + \left(\frac{\partial u}{\partial x}, \frac{\partial w}{\partial x} \right) + \left(v(u) \frac{\partial u}{\partial x}, w \right) + (f_{\beta}(u), w) = 0, \quad \forall w \in W, \quad t \in$$

Following the article, the problem will be formulated based on the Galerkin Method which consists in approximating the problem at hand in a finite-dimensional space. We will consider

1. a partition of Ω into N_{el} equally sized elements Ω_e of length $h = \frac{L}{N_{el}}$ such that $\bar{\Omega} = \bigcup_{e=1}^{N_{el}} \bar{\Omega}_e$ and $\bigcap_{e=1}^{N_{el}} \Omega_e = \emptyset$,
2. $V^h \subset V$ and $W^h \subset W$ the discrete counterparts of V and W ,
3. $W^h = \{\eta^h \in C^0(\Omega); \eta^h|_{\Omega_e} \in P^p \quad \forall \Omega_e, \eta^h|_{\Gamma} = 0\}$ where P^p is the set of interpolation polynomials of degree less or equal to p defined in each element Ω_e .

Thus following the Galerkin method, the weak formulation can then be rewritten as: find $u^h \in V^h, t \in I$, such that

$$\left(\frac{\partial u^h}{\partial t}, w^h \right) + \left(\frac{\partial u^h}{\partial x}, \frac{\partial w^h}{\partial x} \right) + \left(v(u^h) \frac{\partial u^h}{\partial x}, w^h \right) + (f_{\beta}(u^h), w^h) = 0, \quad \forall w^h$$

Regarding the discretisation of time, we will consider $0 = t_0 < t_1 < \dots < t_N$ such that $I_n \equiv (t_n - 1, t_n)$ is a partition of I and $\Delta t^n = t_n - t_{n-1}$.

Following the article, using a θ -scheme (and more specifically a Crank-Nicolson method, i.e. $\theta = 1/2$) to linearise the time derivative, one gets the following final expression of the weak formulation: find $u^h \in V^h, \forall w^h \in W^h, n \in \llbracket 0, N \rrbracket$, such that

$$(u_n^h, w^h) + \frac{\Delta t}{2} \left(\left(\frac{\partial u_n^h}{\partial x}, \frac{\partial w^h}{\partial x} \right) + \left(v(u_n^h) \frac{\partial u_n^h}{\partial x}, w^h \right) + (f_{\beta}(u_n^h), w^h) \right) = (u_{n-1}^h,$$

The finite element functions u^h, w^h are defined as

$$u^h(x, t) = \sum_{j=1}^{Nnp} \phi_j(x) u_{j;n}^h \quad \text{for } x \in \Omega, t \in I_n; \quad (1)$$

$$w^h(x) = \sum_{j=1}^{Nnp} \phi_j(x) w_j^h \quad \text{for } x \in \Omega, \quad (2)$$
(14)

with:

- $u_{j;n}^h$ the value of u^h at the node j at time t_n ,

- w_j^h the corresponding weight,
- Nnp the number of nodal points ($Nel + 1$),
- ϕ_j the spatial shape function for the node j .

Inputting equation (14) into (13) would lead to the creation of a non-linear system. The linearisation will be done using an iterative algorithm comparable to the fixpoint method, which is satisfactory for the diffusive and the convective term, but not for the reaction term $f(u_n^h)$ which is approximated using the Newton Method, i.e. using the tangent to the curve at u_j^k .

$$f(u_j^{k+1}) \sim f(u_j^k) + \left(\frac{df}{du^h} \right)_{|u_j^k} (u_j^{k+1} - u_j^k)$$

$$f(u_n^h) \sim (1 + \beta) (u_j^k)^2 - 2 (u_j^k)^3 + \left[\beta - 2(1 + \beta) u_j^k + 3 (u_j^k)^2 \right] u_j^{k+1}$$

Substituting (14) and the above linearisation into (13) yields the final linearised form.

$$\sum_{i=1}^{Nnp} w_i \left\{ \int_{\Omega} \phi_i \sum_{j=1}^{Nnp} \phi_j u_j^{k+1} dx + \frac{\Delta t}{2} \int_{\Omega} \left[\phi_{i,x} \sum_{j=1}^{Nnp} \phi_{j,x} u_j^{k+1} + \phi_i v(u^k) \sum_{j=1}^{Nnp} \phi_{j,x} u_j^{k+1} \right. \right.$$

We set the following linear system of equations:

$$Mu^{k+1} + K(u^k)u^{k+1} = B(u^k) + C(u_{(n-1)})$$

With M and $K(u^k)$ matrices of size $Nnp \times Nnp$ and $B(u^k)$ and $C(u_{(n-1)})$ vectors of size $Nnp \times 1$, defined as follows:

$$M = \bigsqcup_{e=1}^{Nel} m^e$$

$$K(u^k) = \bigsqcup_{e=1}^{Nel} k^e$$

$$B(u^k) = \bigsqcup_{e=1}^{Nel} b^e$$

$$C(u_{(n-1)}) = \bigsqcup_{e=1}^{Nel} c^e$$

For $i, j = 1, 2$, the element contributions are computed as:

$$m_{ij}^e = \int_{\Omega_e} \left(\phi_i \phi_j + \frac{\Delta t}{2} \phi_{i,x} \phi_{j,x} \right) dx$$

$$k_{ij}^e = \frac{\Delta t}{2} \int_{\Omega_e} \left(v(u^k) \phi_i \phi_{j,x} + \beta - 2(1 + \beta) u^k + 3(u^k)^2 \phi_i \phi_j \right) dx$$

$$b_i^e = -\frac{\Delta t}{2} \int_{\Omega_e} \phi_i \left[(1 + \beta) (u^k)^2 - 2(u^k)^3 \right] dx$$

$$c_{ij}^e = \int_{\Omega_e} \phi_i u_{(n-1)} dx - \frac{\Delta t}{2} \int_{\Omega_e} \left[\phi_{i,x} u_{(n-1),x} + \phi_i v(u_{(n-1)}) u_{(n-1),x} + \phi_i (\beta - (1 + \beta)) \right]$$

The predictor-corrector algorithm applied to (13) is:

```

(1) set u_(0) = u_0
(2) for n=1,...,nstep
    u°=u_(n-1)
    (3) for k=1,...,k_max
        build M, K(u^k), B(u_k) and C(u_(n-1))
        solve for u^(k+1):
            Mu^(k+1) + K(u^k)u^(k+1) = B(u^k) +
C(u_(n-1))
        if max_j=1^Nnp |h_j^(k+1) - u_j^k| =< tol go
to (4)
    end (3)
    (4) u_(n) = u^(k+1)
end (2)

```

The exact solution is found in Petrovskii et al (2003). With

$$\xi = x_i - n_i t + \phi_i, n_i = \sqrt{2}(1 + \beta) - 3\lambda_i, i = 1, 2,$$

$$\lambda_1 = \frac{\beta}{\sqrt{2}}, \quad \lambda_2 = \frac{1}{\sqrt{2}}$$

and $\phi_{1,2}$ as arbitrary constant, one can get the exact solutions which hold true for $\beta \geq 0$. In the case without migration:

$$u(x, t) = \frac{\beta \exp(\lambda_1 \xi_1) + \exp(\lambda_2 \xi_2)}{1 + \exp(\lambda_1 \xi_1) + \exp(\lambda_2 \xi_2)}$$

and in the case with migration:

$$u(x, t) = \tilde{u}(x - at, t) \simeq \frac{\exp\{\lambda_2[x - (n_2 + a)t + \phi_2]\}}{1 + \exp\{\lambda_2[x - (n_2 + a)t + \phi_2]\}}$$

with a the speed of advection (see (10)).

Algorithm encoding

```

In [30]: def initial(x, phi=[100, -100], beta=.2):
    """
    Maps the initial value of u anywhere in space according to the (15) f
    ---
    Arguments:
        x (int/list/ndarr): the variable x
        phi (list/ndarr): the  $\phi_1$  and  $\phi_2$  value, by default 100 and -100
        beta (int): the  $\beta$  value
    ---
    Returns the value of  $u_0(x)$  in a ndarr
    """

```

```

"""
phi = np.array(phi)

xi = np.outer(np.ones_like(phi),x) + phi[:,None]
sq2= np.sqrt(2)
la = np.array([beta/sq2, 1/sq2])[:,None]
ex = np.exp(la*xi)

return (beta*ex[0]+ex[1])/(1+ex[0]+ex[1])

def exact(x, t=0.,phi=[100, -100], beta=.2, mig=False):
    """
    Maps the initial value of u anywhere in space according to the (15) f
    ---
    Arguments:
        x (int/list/ndarr): the variable x
        t (float): the time at which to compute the exact soluti
        phi (list/ndarr): the  $\phi_1$  and  $\phi_2$  value, by default 100 and -100
        beta (int): the  $\beta$  value
        mig (bool): whether or not there is migration
    ---
    Returns the value of u(x,t) in a ndarr
    """
    sq2= np.sqrt(2)
    if not mig:
        c = 0.
    else:
        c = sq2*(beta-1/2)

    phi = np.array(phi)
    la = np.array([beta/sq2, 1/sq2])[:,None]
    ni = sq2*(1+beta) - 3*la
    xi = np.outer(np.ones_like(phi), x) + phi[:,None] - (ni+c)*t
    ex = np.exp(la*xi)

    return (beta*ex[0]+ex[1])/(1+ex[0]+ex[1]) if not mig else ex[1]/(1+ex

# The migration function
def v(u, mig=False):
    """
    The migration function, which can take several parameter values
    ---
    Arguments:
        u (np.ndarray/int): the field u
        mig (bool/list): the values of the migration parameters. There
                        corresponding to the value used in the arti
    ---
    Returns:
        v(u)
    """
    if not mig:
        v0, v1 = 0., 0.
    elif mig is True:
        v0, v1 = .1, -1.
    else:
        v0, v1 = mig[0], mig[1]
    return v0 + u*v1

```

```

# defining each matrix
def mmat(dt=.1):
    @BilinearForm
    def me(u,w,_):
        return u*w+ (dt/2) * dot(u.grad, w.grad)
    return me

def kmat(dt=.1, beta=.2, mig=False):
    @BilinearForm
    def ke(u,w,_):
        uK = _["uk"]
        vu = v(uK, mig=mig)
        return (dt/2)*(w * vu * u.grad[0] + (beta - 2*(1+beta)*uK + 3*uK*
    return ke

def cmat(dt=.1, beta=.2, mig=False):
    @LinearForm
    def ce(w, _):
        uK, preV = _["uk"], _['prev']
        vp = v(preV, mig=mig)
        return (w * preV - (dt/2)*
                (dot(w.grad, preV.grad)
                 + w * vp * preV.grad[0]
                 + w * (beta*preV - (1+beta)*preV**2 + preV**3)))
    return ce

def bmat(dt=.1, beta=.2):
    @LinearForm
    def be(w,_):
        uK = _["uk"]
        return -(dt/2)*w*((1+beta)*uK**2 - 2*uK**3)
    return be

# creating the universe
def basis(limit=500, Nel=400, rAll=False):
    Nnp = Nel + 1
    univ = np.linspace(-limit, limit, Nnp)
    mesh = fem.MeshLine(univ)
    elem = fem.ElementLinePl()
    basis= fem.Basis(mesh, elem)
    if rAll:
        return basis, univ, mesh, elem
    return basis

```

```

In [ ]: def pc_algo(base=None,
                init="initial",
                phi=[100,-100],
                limit=500,
                Nel=400,
                nstep=20,
                kmax=100,
                dt=.1,
                beta=.2,
                betai=None,
                mig=False,
                tol=10E-3,
                verbose=False):
    """
    the function to call, it should work with just pc_algo()
    ---

```

Arguments:

base (NoneType/fem.CellBasis): the basis to use for the mesh
init (string/list/np.ndarray): the repartition of u over Ω at in
phi (list/np.ndarray): the ϕ_1 and ϕ_2 value, by default 1
limit (int): the limits of Ω
Nel (int): the number of elements in which Ω
out_bc (list): the outer conditions to abide by
nstep (int): the maximum number of time-steps
kmax (int): the maximum number of iterations
dt (float): the time step-size
beta (float): the intensity of the Alle effect
mig (bool/list): the migration parameters
tol (float): the tolerance of the iterative pr
verbose (bool): whether to print the result

Returns

un (np.ndarray): returns $u(x,t) \forall (x,t) \in (\Omega_h \times I_h)$

"""

(1)

if base is None:

base = basis(limit, Nel)

if init == "initial":

if betai is None:

un = np.array([initial(base.doflocs[0], phi=phi, beta=beta)])

else:

un = np.array([initial(base.doflocs[0], phi=phi, beta=betai)])

elif init == "linear":

lin = np.array(np.linspace(0, 1, 51))

zero = np.array([0]*175)

uns = np.array([1]*175)

un = np.array([np.concatenate((zero, lin, uns))])

elif init == "sinusoidal":

Time = np.linspace(-5, 5, 25)

Amplitude = 0.5

Period = 10

sin = Amplitude * np.sin(np.pi * Time / Period)

sin += 0.5

apply setting to zero where the boolean mask is True

zero = np.array([0]*188)

uns = np.array([1]*188)

un = np.array([np.concatenate((zero, sin, uns))])

else:

un = np.array(init)

dof_out = base.get_dofs(lambda x: (x[0] == -limit) | (x[0] == limit))

dof_in = base.complement_dofs(dof_out)

u_bc = np.zeros(base.N)

out_bc = (np.round([un[0][0], un[0][-1]]))

u_bc[dof_out] = np.array(out_bc)

m0 = mmat(dt)

kl = lambda uk: kmat(dt, beta, mig)

c0 = cmat(dt, beta, mig)

b0 = bmat(dt, beta)

M = asm(m0, base)

(2)

```

for n in range(1,nstep+1):
    prev = un[n-1]
    uk = prev.copy()

    # (3)
    for k in range(0,kmax):
        K = asm(kl(uk), base, uk=uk)
        C = asm(c0, base, uk=uk, prev=prev)
        B = asm(b0, base, uk=uk)

        left, right, *_ = fem.condense(M+K, C+B, D=dof_out, x=u_bc)

        new = np.zeros(base.N)
        new[dof_in] = spsolve(left, right)
        new[dof_out] = u_bc[dof_out]

        # (4)
        if np.linalg.norm(new - uk, np.inf) < tol:
            print(f"Converged at iteration #{k}/{kmax}") if (k>1 and
            break
        elif verbose and k==kmax-1:
            print(f"Warning! Still no convergence at step {kmax}")
            uk = new.copy()

    un = np.append(un, [new], axis=0)

    if n%100 == 0 and verbose:
        print(f"Step {n}/{nstep}")
return un

```

Numerical experiments

```

In [7]: def plotting(U, base=None, nstep=2400, dt=.1, every=400, xbounds=250, tit
"""
Plot the evolution of u(x,t) over space for selected timesteps.
---
Arguments:
    U          (np.ndarray): ndarray (time x space) from pc_algo
    base       (fem.CellBasis): scikit-fem Basis (for x coordinates)
    solutions  (np.ndarray): (time x space) from pc_algo
    nstep      (int): total number of steps
    dt         (float): timestep size
    every      (int): plot every N steps
    xbounds    (int): the bounds to use for xlim
    title      (str): optional title for the plot
"""
nplot = np.arange(0, nstep + 1, every)
if base is None:
    base = basis(limit, Nel)
x = base.doflocs[0]

fig, ax = plt.subplots(figsize=(12, 7))
for n in nplot:
    ax.plot(x, U[n], label=f"t = {n * dt:.1f}")

ax.legend()
ax.set_xlabel("x")
ax.set_xlim(-xbounds, xbounds)

```

```

ax.set_ylabel("u(x,t)")
ax.set_title(title or f"Solution evolution (every {every} steps)")
ax.grid(True)
fig.tight_layout()
return ax

```

```

In [8]: base = basis()
dt = .1

```

Fig 1. Spatial distribution of the population density at different moments for $\beta = 0.2$, $v_0 = v_1 = 0$.

```

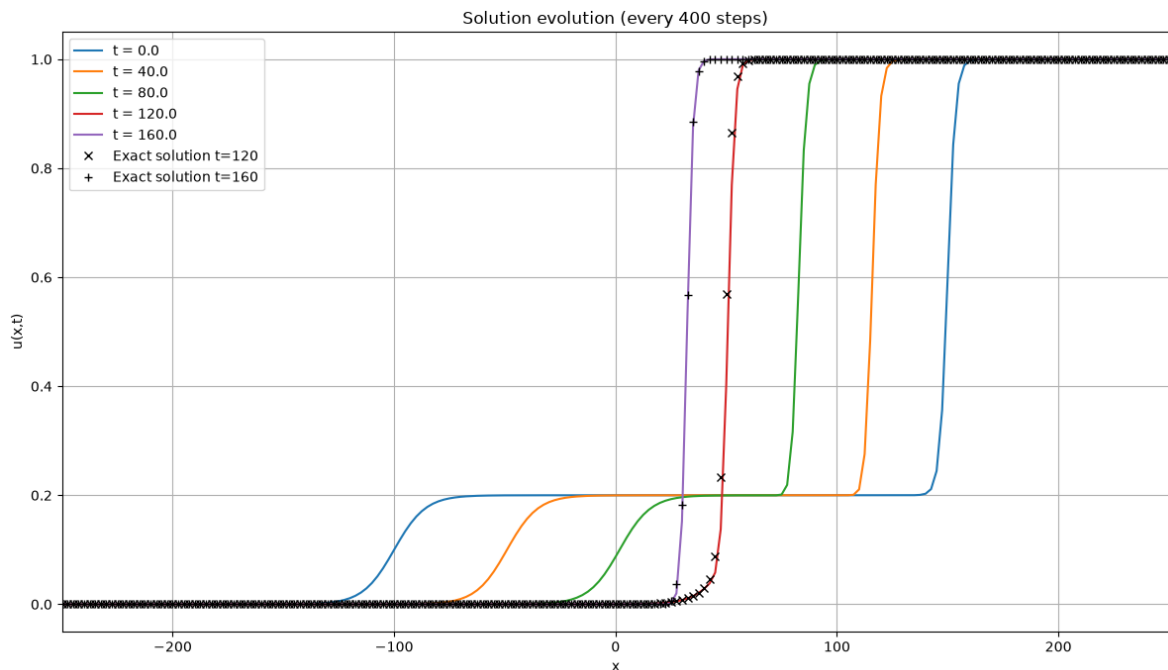
In [9]: nstep=1600
solutions1 = pc_algo(base=base,nstep=nstep,dt=dt)

```

```

In [10]: ax = plotting(solutions1, base, nstep)
x = base.doflocs[0]
ax.plot(x, exact(x, 120), 'kx', label='Exact solution t=120')
ax.plot(x, exact(x, 160), 'k+', label='Exact solution t=160')
ax.legend()
plt.show()

```



The initial condition is set with $\beta = 0.2$, $\phi_1 = 100$ and $\phi_2 = -100$. The numerical solution shown here is for a strong Allee effect with $\beta = 0.2$ and $v_0 = v_1 = 0$

Fig 2. Spatial distribution of the population density at different moments for $\beta = 0.2$, $v_0 = 0.1$, $v_1 = -1.0$.

```

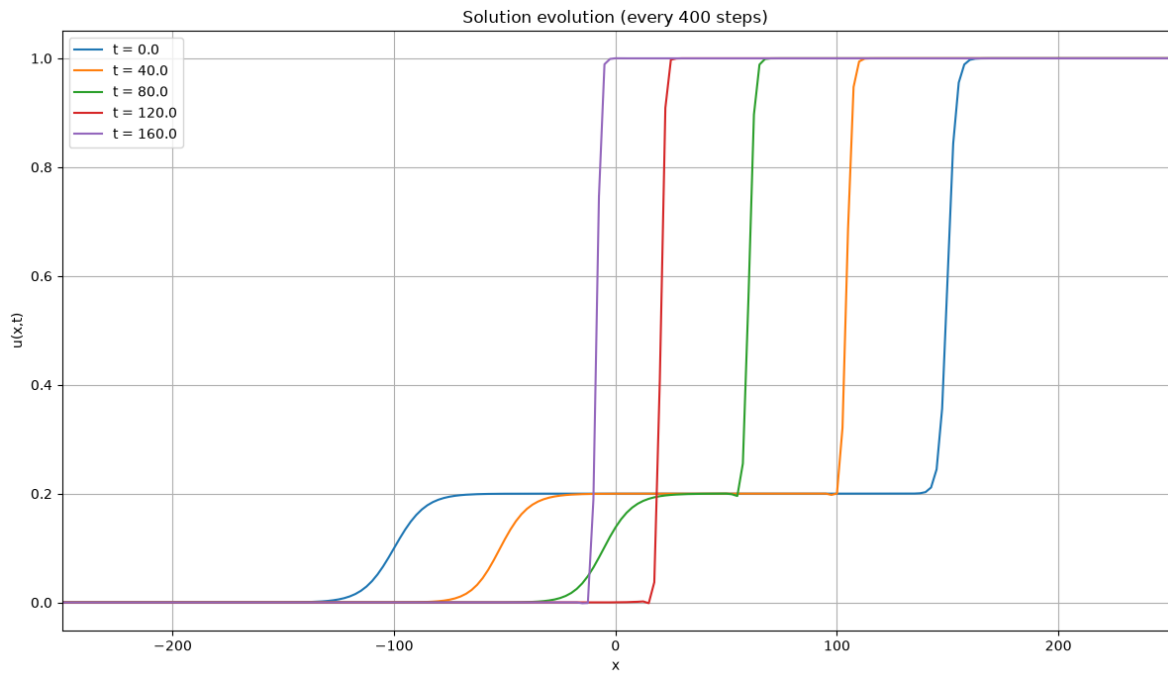
In [14]: nstep=1600
solutions2 = pc_algo(base=base,nstep=nstep,dt=dt,mig=[0.1, -1])

```

```

In [15]: ax = plotting(solutions2, base, nstep)
x = base.doflocs[0]

```



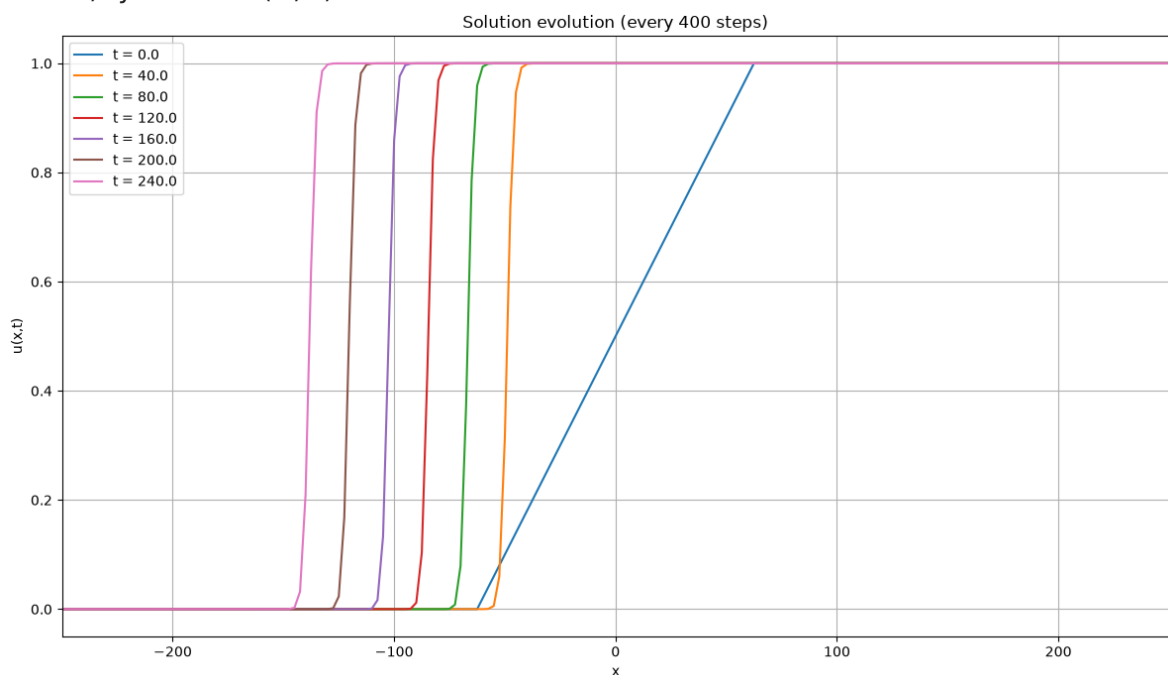
The initial condition is the same as before and we are still in a strong allee effect setting but with $v_0 = 0.1$ and $v_1 = -1.0$ and we see that the front are much more spaced.

Fig 3. Spatial distribution of the population density at different moments for $\beta = 0.2$, $v_0 = v_1 = 0$ (linear initial condition).

```
In [16]: nstep=2400
solutions3 = pc_algo(base=base,nstep=nstep,dt=dt, init="linear")
```

```
In [17]: plotting(solutions3, base, nstep)
```

```
Out[17]: <Axes: title={'center': 'Solution evolution (every 400 steps)'}, xlabel=
'x', ylabel='u(x,t)'>
```



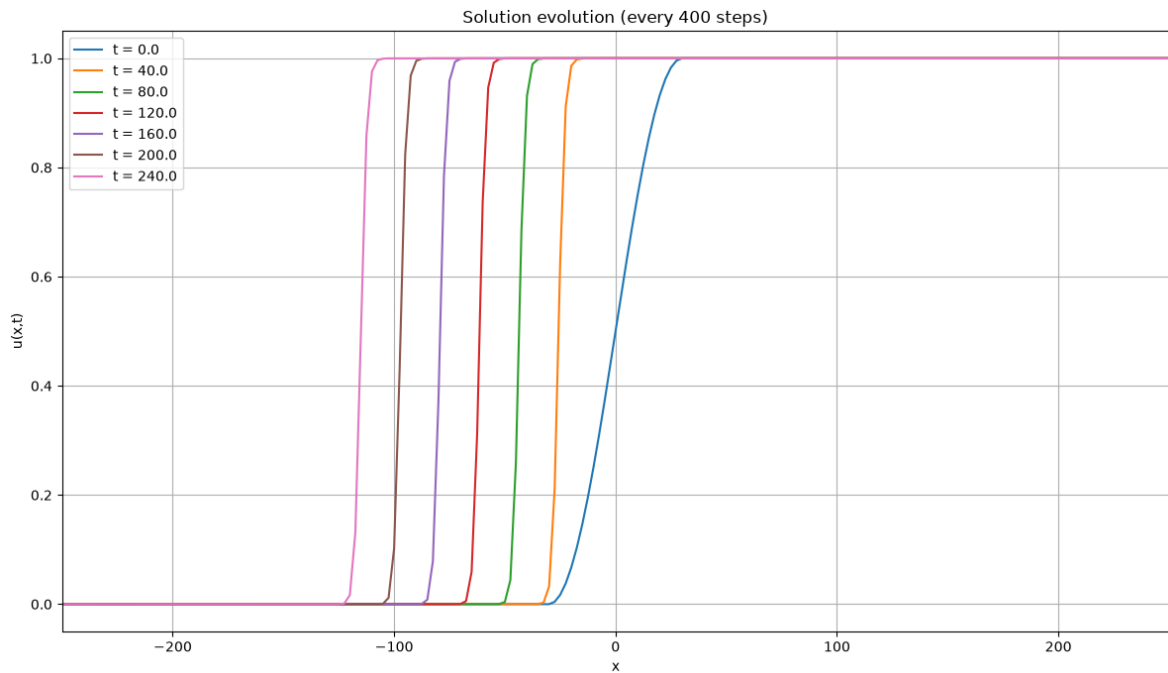
With a linear initial condition, the travelling population front is formed quickly but otherwise the behavior is similar to before.

Fig 4. Spatial distribution of the population density at different moments for $\beta = 0.2, v_0 = v_1 = 0$ (sigmoidal initial condition).

```
In [18]: nstep=2400
solutions4 = pc_algo(base=base,nstep=nstep,dt=dt, init="sinusoidal")
```

```
In [19]: plotting(solutions4, base, nstep)
```

```
Out[19]: <Axes: title={'center': 'Solution evolution (every 400 steps)', xlabel=
'x', ylabel='u(x,t)'}>
```



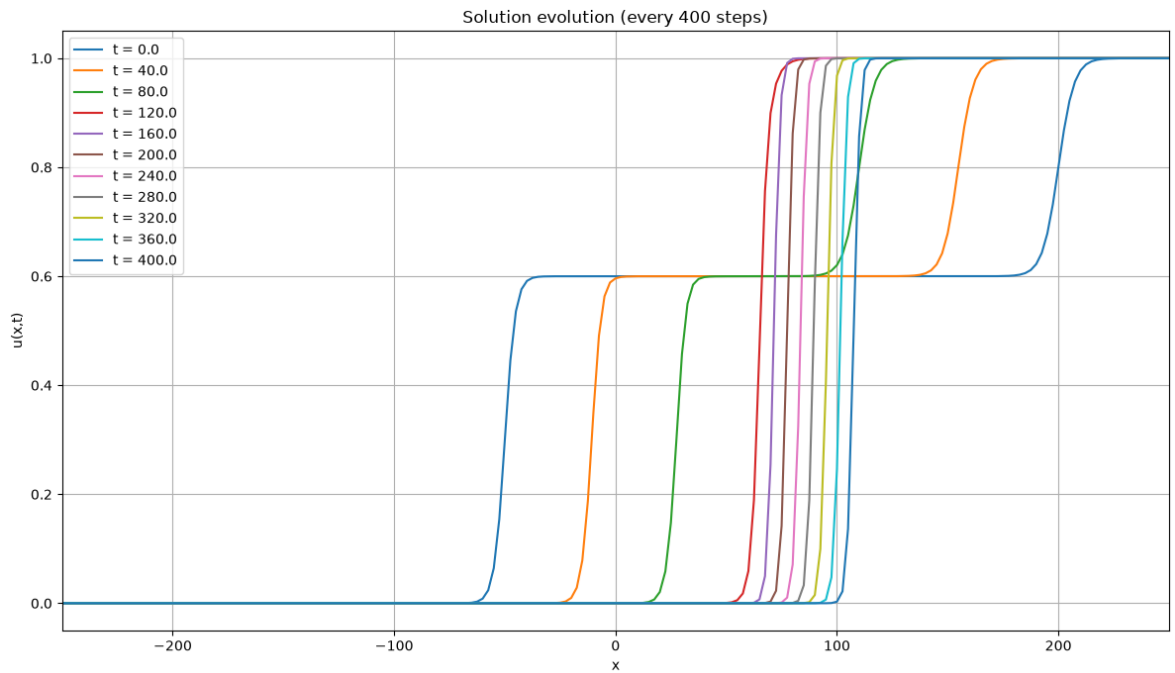
The same conclusion can be drawn with a sinusoidal initial condition.

Fig 5. Spatial distribution of the population density at different moments for $\beta = 0.6, v_0 = v_1 = 0$.

```
In [20]: nstep=4000
solutions5 = pc_algo(base=base,phi=[50,-50],beta=.6,nstep=nstep)
```

```
In [21]: plotting(solutions5, base, nstep)
```

```
Out[21]: <Axes: title={'center': 'Solution evolution (every 400 steps)', xlabel=
'x', ylabel='u(x,t)'}>
```



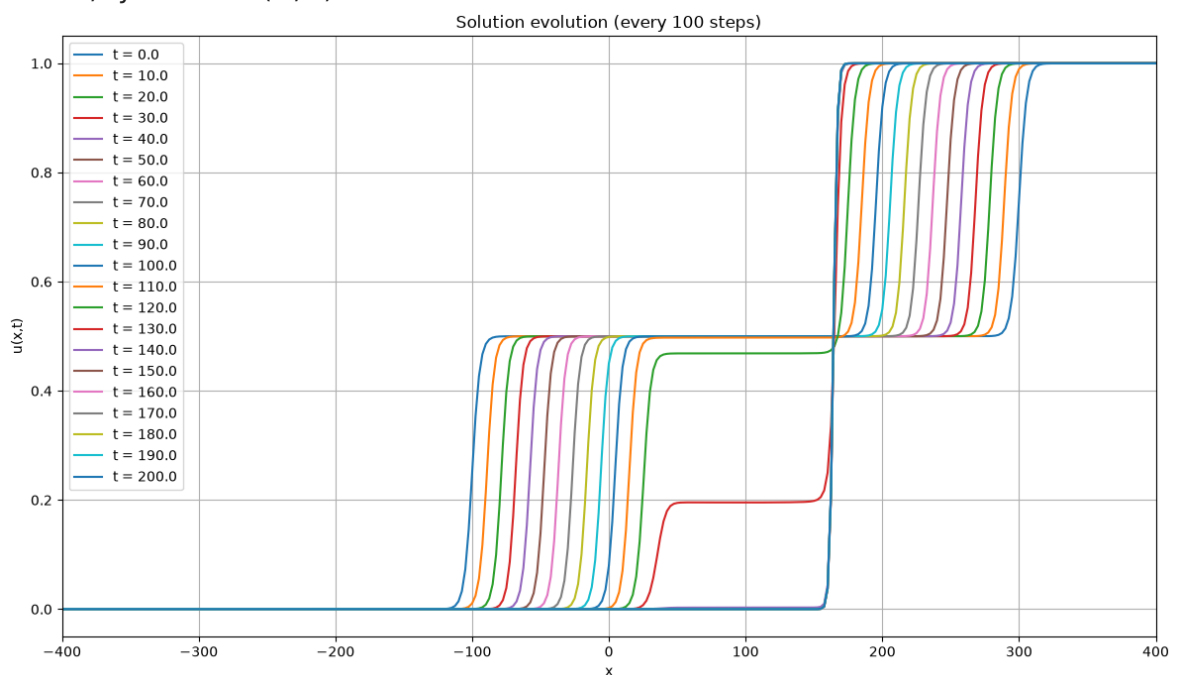
This time with a stronger Allee effect ($\beta = 0.6$) and an initial condition with $\phi_1 = 50$ and $\phi_2 = -50$, we observe a retreat phenomenon.

Fig 6. Spatial distribution of the population density at different moments for $\beta = 0.5$, $v_0 = v_1 = 0$.

```
In [22]: nstep=2000
solutions6 = pc_algo(base=base,beta=.5,nstep=nstep)
```

```
In [23]: plotting(solutions6, base, nstep, every=100, xbounds=400)
```

```
Out[23]: <Axes: title={'center': 'Solution evolution (every 100 steps)'}, xlabel=
'x', ylabel='u(x,t)'>
```



There is a β for which the wave front becomes stationary. With $\phi_1 = 100$ and $\phi_2 = -100$, for $\beta = 0.5$, from $t = 140$, the wave front doesn't change anymore.

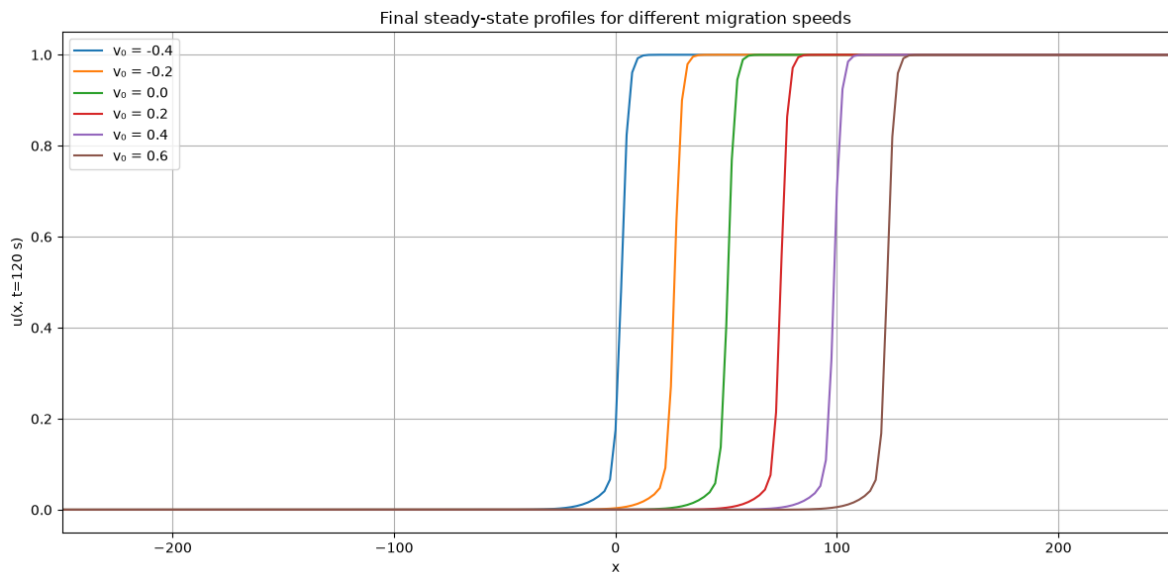
Fig 7. Spatial distribution of the population density at $t = 120$ for $\beta = 0.2$ and $v_1 = 0$.

```
In [34]: v0s = np.arange(-0.4, 0.7, 0.2)
nstep=1200
solutions7 = []
for v0 in v0s:
    sol = pc_algo(base=base, beta=.2, mig=[v0, 0], nstep=nstep)
    solutions7.append(sol[-1])

In [35]: plt.figure(figsize=(12, 6))
x = base.doflocs[0]

for i, v0 in enumerate(v0s):
    plt.plot(x, solutions7[i], label=f"v0 = {v0:.1f}")

plt.xlabel("x")
plt.ylabel("u(x, t=120 s)")
plt.xlim(-250, 250)
plt.title("Final steady-state profiles for different migration speeds")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```



We confirm that with bigger v_0 , the travelling front speed is decreased and can even lead to a retreat phenomenon.

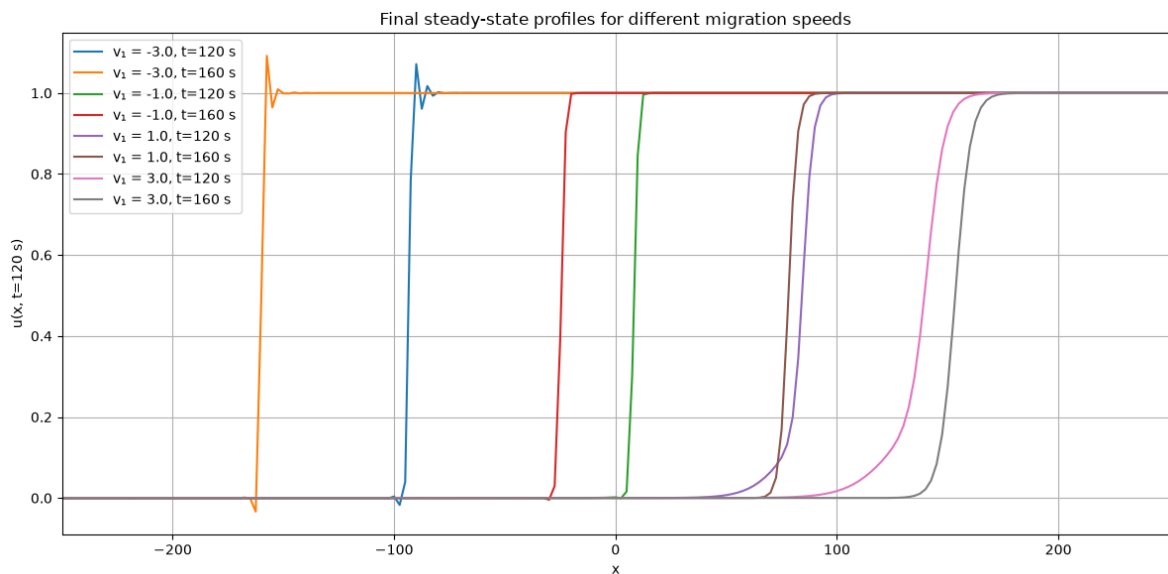
Fig 8. Spatial distribution of the population density at $t = 120$ and $t = 160$ for $\beta = 0.2$ and $v_0 = 0$.

```
In [36]: v1s = np.arange(-3., 3.1, 2)
nstep1=1200; nstep2=1600
solutions8120 = []
solutions8160 = []
for v1 in v1s:
    sol1 = pc_algo(base=base, beta=.2, mig=[0, v1], nstep=nstep1)
    sol2 = pc_algo(base=base, beta=.2, mig=[0, v1], nstep=nstep2)
    solutions8120.append(sol1[-1]); solutions8160.append(sol2[-1])
```

```
In [37]: plt.figure(figsize=(12, 6))
x = base.doflocs[0]

for i, v1 in enumerate(v1s):
    plt.plot(x, solutions8120[i], label=f"v1 = {v1:.1f}, t=120 s")
    plt.plot(x, solutions8160[i], label=f"v1 = {v1:.1f}, t=160 s")

plt.xlabel("x")
plt.ylabel("u(x, t=120 s)")
plt.xlim(-250, 250)
plt.title("Final steady-state profiles for different migration speeds")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```



Here, we can observe that what we said with v_0 also holds for v_1 .

Fig 9. Spatial distribution of the population density at $t = 120$ and $t = 160$ for $\beta = 0.2$ and $v_0 = 0$ with the SUPG formulation.

Due to the complexity of SUPG formulation and the deadline drawing near, we did not include this figure as it is not essential for the study.

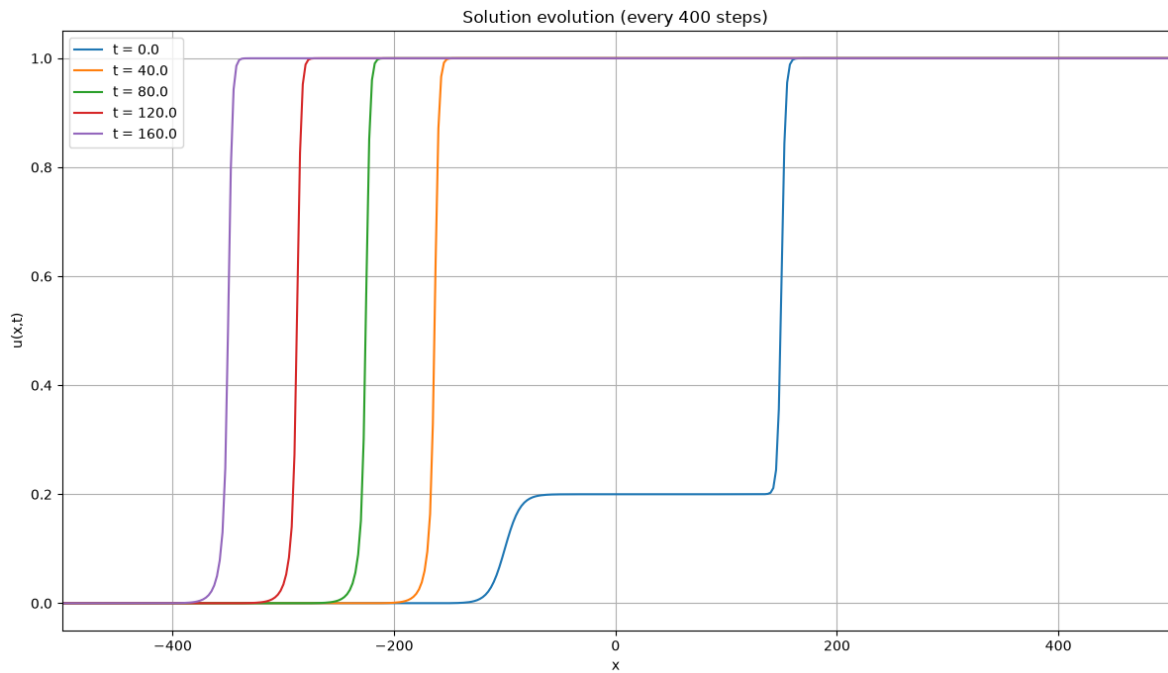
Fig 10. Spatial distribution of the population density at different moments for $\beta = -0.2$, $v_0 = v_1 = 0$.

```
In [28]: nstep=1600

solutions10 = pc_algo(base=base,nstep=nstep,beta=-.2,dt=dt, betai=.2)
```

```
In [29]: plotting(solutions10, base, nstep, xbounds=500)
```

```
Out[29]: <Axes: title={'center': 'Solution evolution (every 400 steps)'}, xlabel=
'x', ylabel='u(x,t)'>
```



We now start to study the weak Allee effect with $\beta = -0.2$. The initial condition however is still calculated with $\beta = 0.2$. When compared to Fig1, we note that the speed of the travelling front is way higher with $\beta < 0$.

Fig 11. Spatial distribution of the population density at different moments for $\beta = -0.1, v_0 = v_1 = 0$.

```
In [30]: nstep=1600
solutions11 = pc_algo(base=base,nstep=nstep,beta=-.1,dt=dt, betai=.2)
```

```
In [31]: plotting(solutions11, base, nstep, xbounds=500)
```

```
Out[31]: <Axes: title={'center': 'Solution evolution (every 400 steps)'}, xlabel='x', ylabel='u(x,t)'>
```

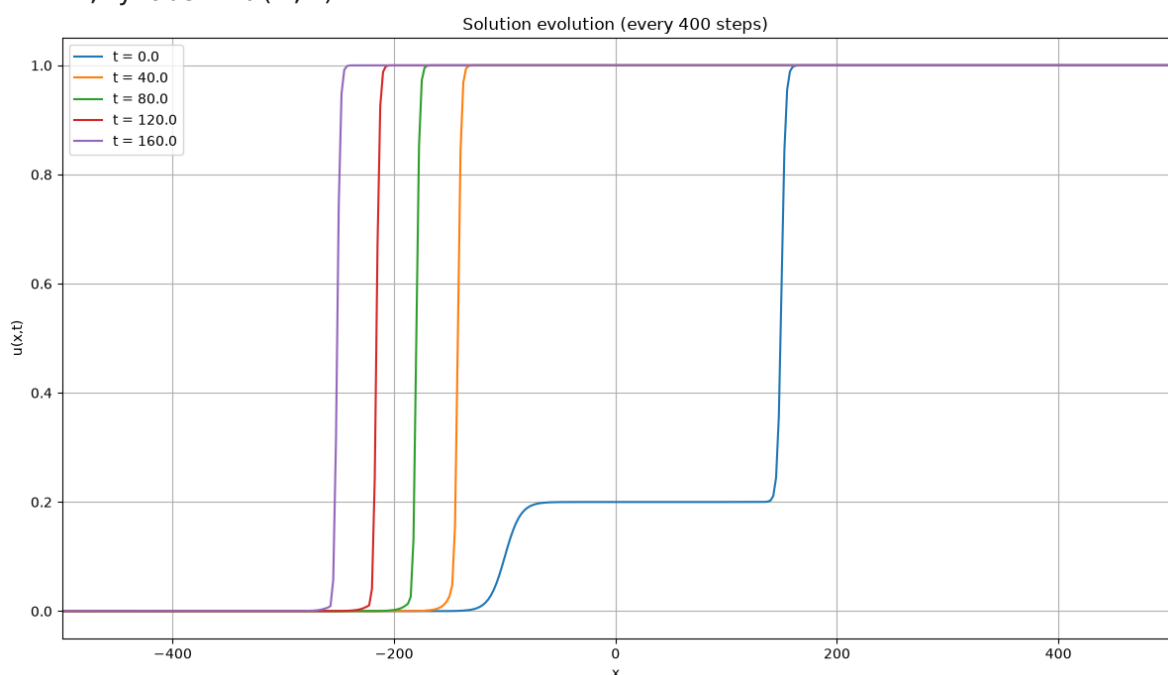


Fig 12. Spatial distribution of the population density at different

moments for $\beta = -0.3, v_0 = v_1 = 0$.

```
In [32]: nstep=1600  
solutions12 = pc_algo(base=base,nstep=nstep,beta=-.3,dt=dt, betai=.2)
```

```
In [33]: plotting(solutions12, base, nstep, xbounds=500)
```

```
Out[33]: <Axes: title={'center': 'Solution evolution (every 400 steps)', xlabel=  
'x', ylabel='u(x,t)'}>
```

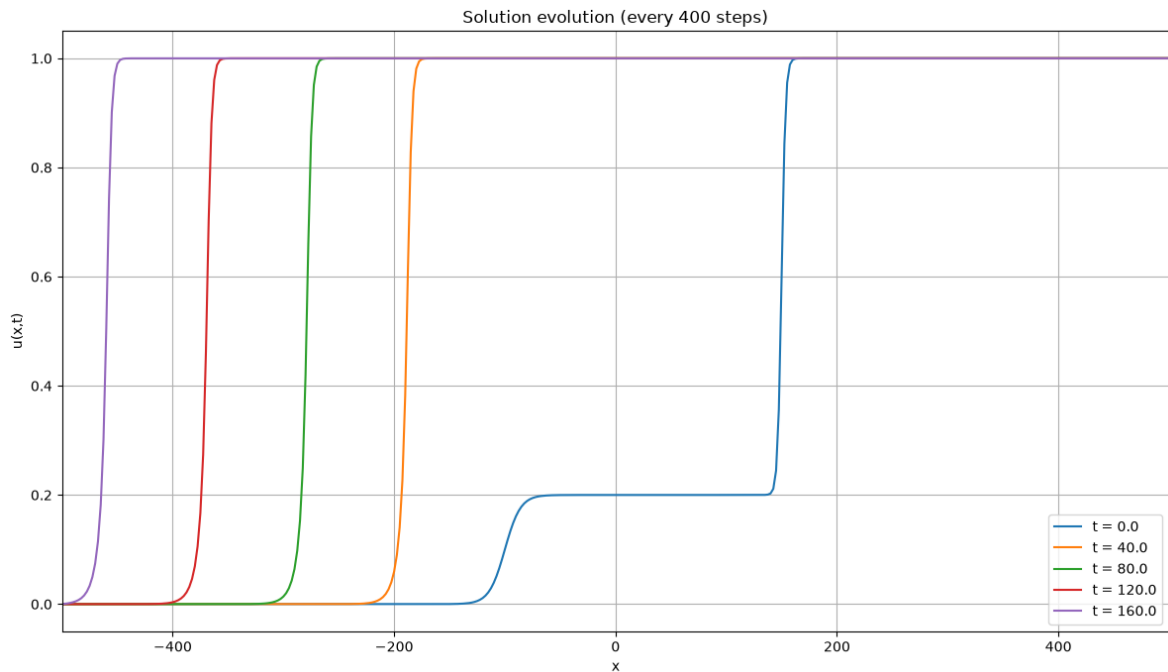


Fig 11 and 12 illustrate that with smaller Allee effect the front speed increase.

Fig 13. Spatial distribution of the population density at different moments for $\beta = -0.2, v_0 = 0.1$ and $v_1 = 0$.

```
In [21]: nstep=1600  
solutions13 = pc_algo(base=base,nstep=nstep,beta =-.2,dt=dt,mig=[0.1, 0],
```

```
In [22]: plotting(solutions13, base, nstep, xbounds=500);
```

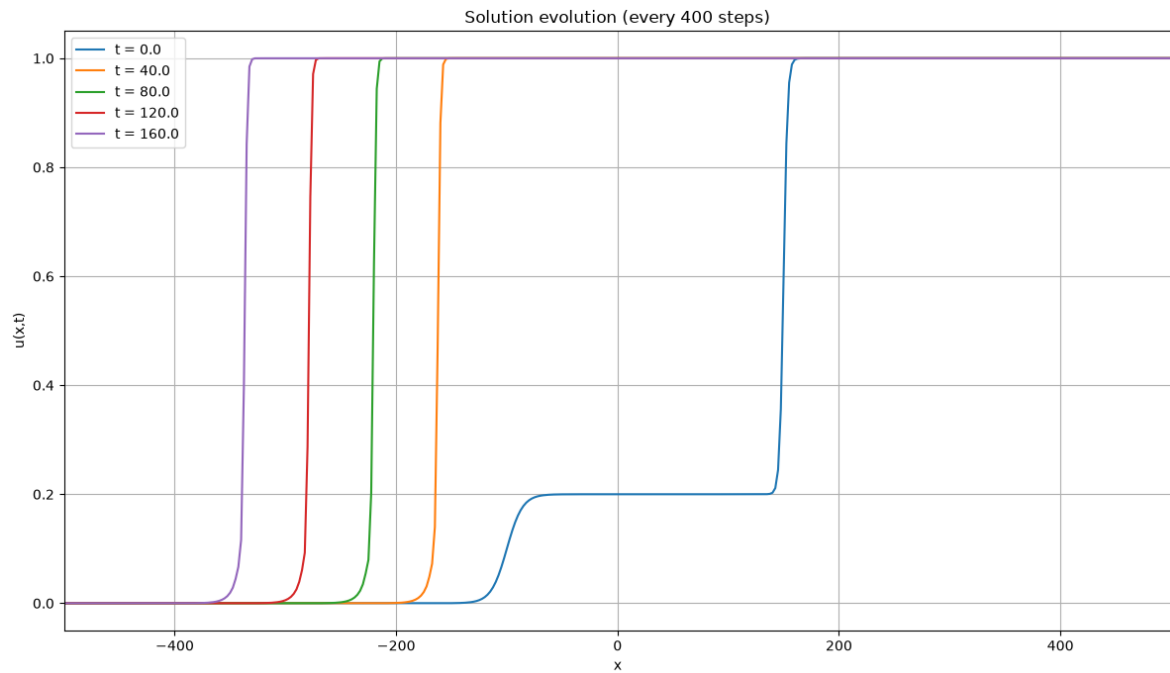


Fig 14. Spatial distribution of the population density at different moments for $\beta = -0.2$ (initial solution).

```
In [11]: nstep=1600
solutions14 = pc_algo(base=base,nstep=nstep,beta = -.2,dt=dt,mig=True, phi
```

```
In [ ]: plotting(solutions14, base, nstep, xbounds=500);
```

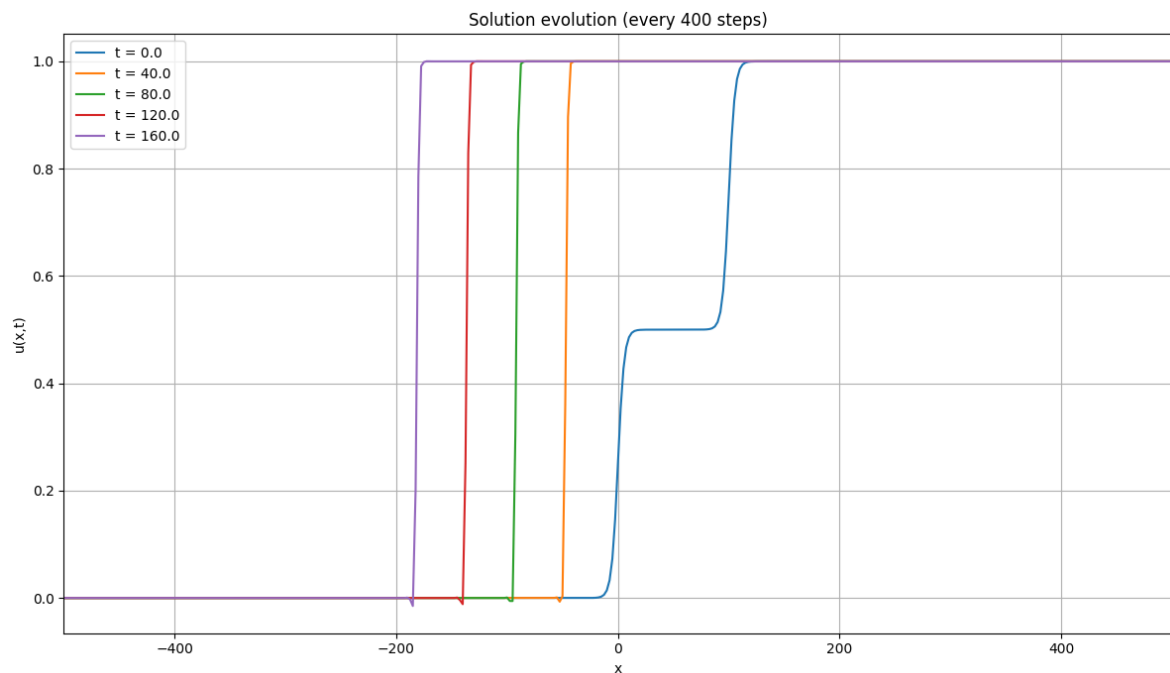
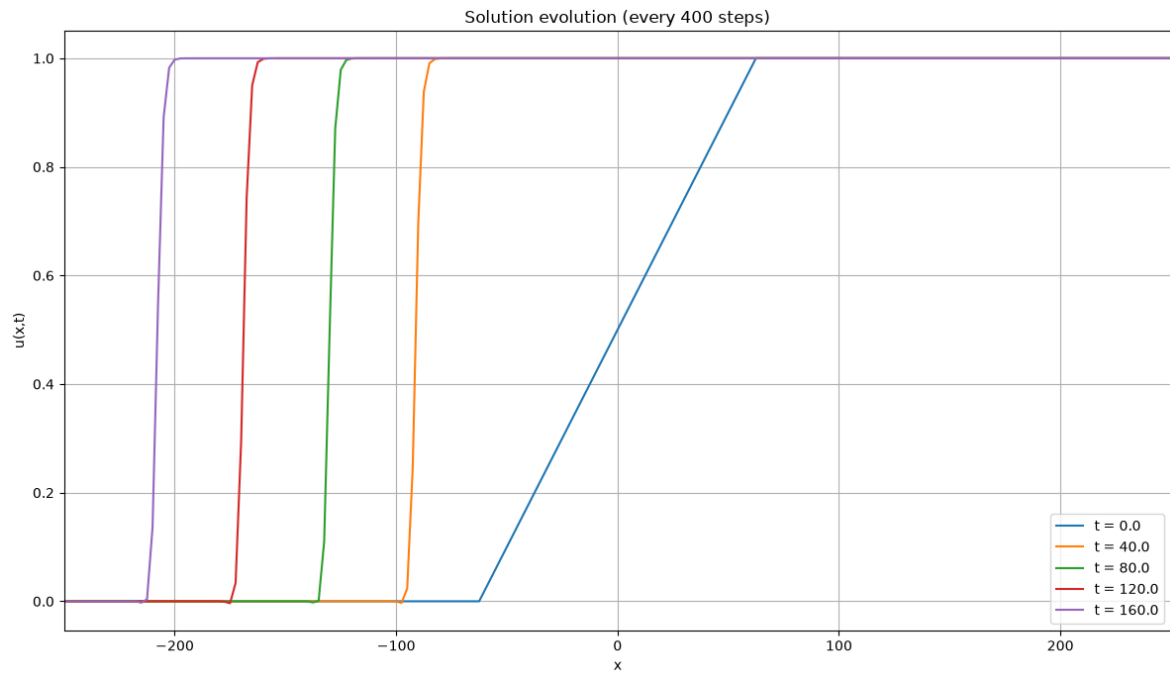


Fig 15. Spatial distribution of the population density at different moments for $\beta = -0.2$ (linear initial solution).

```
In [13]: nstep=1600
solutions15 = pc_algo(base=base,nstep=nstep,dt=dt, init="linear", beta=-.
```

```
In [17]: plotting(solutions15, base, nstep, xbounds=250);
```



References

- [1] T. Gustafsson and G. D. McBain, 'scikit-fem: A Python package for finite element assembly', *Journal of Open Source Software*, vol. 5, no. 52, p. 2369, 2020, doi: 10.21105/joss.02369.
- [2] H. T. Banks, P. M. Kareiva, and L. Zia, 'Analyzing Field Studies of Insect Dispersal Using Two-Dimensional Transport Equations', *Environ Entomol*, vol. 17, no. 5, pp. 815–820, Oct. 1988, doi: 10.1093/ee/17.5.815.
- [3] A. Okubo and S. A. Levin, *Diffusion and Ecological Problems: Modern Perspectives*, second ed., vol. 14. in *Interdisciplinary Applied Mathematics*, pp. 304–317. Springer Science & Business Media, 2002.
- [4] M. A. Lewis and P. Kareiva, 'Allee Dynamics and the Spread of Invading Organisms', *Theoretical Population Biology*, vol. 43, no. 2, pp. 141–158, Apr. 1993, doi: 10.1006/tpbi.1993.1007.
- [5] S. Petrovskii and B.-L. Li, 'An exactly solvable model of population dynamics with density-dependent migrations and the Allee effect', *Mathematical Biosciences*, vol. 186, no. 1, pp. 79–91, Nov. 2003, doi: 10.1016/S0025-5564(03)00098-1.